Processes Management

A *process* is any program, application, or command that runs on the system. A process is created in memory in its own address space when a program, application, or command is initiated. Processes are organized in a hierarchical fashion. Each process has a *parent process* (a.k.a. a *calling process*) that spawns it. A single parent process may have one or many *child processes* and passes many of its attributes to them at the time of their creation. Each process is assigned a unique identification number, known as the *process identifier* (PID), which is used by the kernel to manage and control the process through its lifecycle. When a process completes its lifecycle or is terminated, this event is reported back to its parent process, and all the resources provisioned to it are then freed and the PID is removed.

A major distinction can be made between two process types:

a) Shell jobs are commands started from the command line. They are associated with the shell that was current when the process was started. Shell jobs are also referred to as interactive processes.

b) Daemons are processes that provide services. They normally are started when a computer is booted and often (but certainly not in all cases) they are running with root privileges.

C) When a process is started, it can use multiple threads. A thread is a task started by a process and that a dedicated CPU can service. The Linux shell offers tools to manage individual threads. Thread anagement should be taken care of from within the command.

How to differentiate between process types ?

If you execute the (ps aux) command you can find one of the following:

a. Shell Jobs: is command you run (I think it's obvious to notice).
b. Daemons: is a process with the end (d) letter like httpd, vsftpd, dhcpd,……etc.
c. Kernel threads: with square brackets like [kthreadd].

Managing Shell Jobs

the job is started as a foreground process, occupying the terminal it was started from until it has finished its work. As a Linux administrator, you need to know how to start shell jobs in the foreground or background and what can be done to manage shell jobs.

If you know that a job will take a long time to complete, you can start it with an **&** behind it. Like

firefox &

#nano &

gedit &

To display all process from background

jobs

To restore process from background to foreground

fg %<Index-of-process>

fg %1 for process with id 1.

To terminate process

CTRL + C

Managing Parent Child Relations

Every process has a parent. If you call firefox over shell then shell is the parent and firefox is he child.

Foreground

If you kill (terminate) the shell the firefox will be killed either.

Background

If you kill (terminate) the shell the firefox still running because the parenthood return to systemd.

System is parent of all process with pid (1) and it's parent is kernel (0).

ps aux to display process ID and more

ps -ef to display ppid of service

pidof <service> to display pid of service

#pgrep <service> to display pid of service

KILL Command

Kernel send a signal to a service

kill -l to list all signals of kill cmd.

Types of signals:

a. SIGHUB 1 send a signal to make service reloaded
b. SIGKILL 9 send signal force service to be terminated
c. SIGTERM 15 send signal to make service terminated after doing it's job (default signal)

#kill <PID-OF-SERVICE>

Adjusting Process Priority with nice

The default niceness of a process is set to 0.

Highest priority -20 and the lowest priority 19.

nice -n <PRIORITY> <SERVICE-NAME>

Ex.

nice -n -19 firefox

#renice -n <PRIORITY> <PID-OF-SERVICE>

Ex.

 # renice -n 10 -p 1234

Using top to Manage Processes

Top is so great because it gives an overview of the most active processes currently running.

You can use (r) command from the **top** utility to change the priority of a currently running process or any process with it's id.

You can use (k) command from the **top** utility to kill the currently running process or any process with it's id.

ENG. Muhammad Adel